Chapter 1

Hardware and Software Installation Notes

Chapter 2

Troubleshooting Software Bringup

Chapter 3

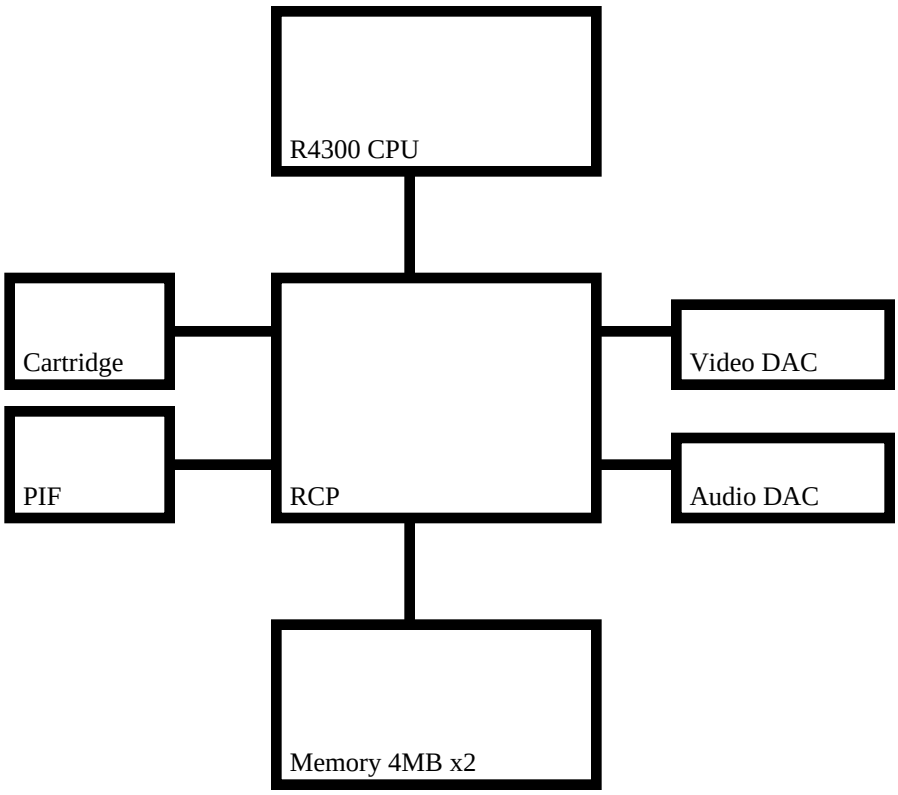
Hardware Architecture

This chapter describes the hardware architecture of the Nintendo 64 game machine, in order to help you write software for the machine. Later sections of this manual describe the details you need to know to program each component.

The Nintendo 64 game consists of a number of hardware components that work together to produce the graphics and audio for the game. The heart of the system is the Reality Coprocessor (RCP). Attached to the RCP are memory chips, the MIPS R4300 CPU, and some miscellaneous I/O chips.

The RCP is the center of the game; all data must pass through it. It acts as the memory controller for the CPU. The RCP runs the graphics and audio microcode. The display portion of the RCP renders into the graphics framebuffer located in main memory. The video and audio portions of the RCP, DMA framebuffer, and audio data from main memory to drive the video and audio DACs. Figure 3-1, "Nintendo 64 Hardware Block Diagram," on page 42 is a block diagram of the Nintendo 64 system.

Figure 3-1 Nintendo 64 Hardware Block Diagram



Execution Overview

The CPU and RCP are both processors that can execute at the same time. Threads execute on the CPU and tasks execute on the RCP. Accesses to main memory from threads and tasks also occurin parallel.

The game program runs on the R4300 CPU as a collection of threads, each of which has its own stack. The operating system is a collection of routines that

can be called in a thread. The operating system controls which thread is running on the CPU. A thread can access all of physical memory. See Chapter 6, "Operating System Overview," for more information.

Tasks run on the RCP, which is a microcode engine that processes a task list. Task lists are generated by a thread running on R4300 CPU and are stored in main memory. The game program creates the task list, calls an OS routine to load the appropriate microcode, and then starts the RCP running to process the task list. The microcode on the RCP reads the task list from main memory. The RCP task can also write into main memory.

RCP: Reality Coprocessor

The RCP is really a collection of processors, memory interfaces, and control logic. The Reality Signal Processor (RSP) is the microcode engine that executes audio and graphics tasks. The Reality Display Processor (RDP) is the graphics display pipeline that renders into the framebuffer. The memory interfaces provide access to main memory for the CPU, RSP, RDP, video interface, audio interface, peripherial devices, and serial game controllers. It is very important to remember that these interfaces may be active at the same time and that the RSP and RDP are running in parallel.

RSP: Reality Signal Processor

The RSP is the processor used by the graphics and audio microcode. The RSP consists of a Scalar Unit (SU), a Vector Unit (VU), instruction memory (IMEM), and data memory (DMEM). The microcodeis fetched from EMEM and has direct access to DMEM. The RSP can also access main memory using DMA. All memory references in the RSP are physical. However, the microcode uses a segment address table to translate segmented addresses provided in the task lists into physical addresses. The IMEM and DMEM are both 4 KB. The SU implements a subset of the R4000 instruction set. The VU has eight 16-bit elements.

For information on how the RSP is used to imlemenl part of the graphics pipeline, see Chapter 12, "RSP Graphics Programming", Chapter 19, "The Audio Library," describes how the RSP is used in audio processing

RDP: Reality Display Processor

The RDP is the graphics display pipeline that executes an RDP display list generated by the RSP and CPU. The RDP consists of a Rasterizer (RS), a Texture Unit (TX), 4 KB of texture memory (TMEM), a Texture Filter Unit (TF), a Color Combiner (CC), a Blender (BL), and a Memory Interface (MI).

The RS rasterizes triangles and rectangles. The TX samples textures loaded in TMEM. The TF filters the texture Samples. The CC combines and interpolates between two colors. The BL blends the resulting pixels with pixels in the frame buffer and performs z-buffer and ani-aliasing operations. The MI performs the read, modify, and write operations for the individual pixels at either one pixel per clock or one pixel for every two clocks. The MI also has special modes for lading the TMEM, filling rectangles (fast clears), and copying multiple pixels from the TMEM into the framebuffer (sprites).

The RDP accesses main memory using physical addresses to load the internal TMEM, to read the framebuffer for blending, to read the z-buffer for depth comparison, and to write the z and framebuffers. The microcode on the RSP translates the segmented addresses in the task list into physical addresses.

The global state registers are used by all stages of the pipeline. There are a number of sync commands to provide synchronization. For example, a pipe async is used before changing one of the rendering modes. This ensures that all previous rendering affected by the mode change occurs before the mode change.

The command list for the RDP usually comes directly from the RSP. However, it is possible to feed the RDP pipeline from a command list that has been stored in main memory. See Chapter 13, "RDP Programming" for more information on the RDP.

Nintendo 64 game runs in kernel mode with 32-bit addressing. 64-bit integer operations are available in this mode. However, the 32-bit C calling convention is used to maximize performance.

For more information on the R4300 and the operating system control of the CPU see the MIPS Microprocessor R4000 User’s Manual and Chapter 6, "Operating System Overview".

Memory Issues

The main memory in the system is used in parallel by the R4300 CPU, the RSP microcode engine, the RDP graphics pipeline, and the other I/O interfaces of the RCP. The software is responsible for defining the memory map. See Chapter 9, "Basic Memory Management" for more details.

Addressing

The R4300 CPU can use physical or virtual addresses. The TLB maps virtual addresses into physical addresses. It is anticipated that programs will mainly use KSEG0 (cached, unmapped) addresses for instructions and data. The RSP hardware uses physical addresses. The microcode imposes a segmented addressing scheme to generate the physical addresses. Bits 24 through 27 of the segmented address are used to index into a 16-entry table to obtain the base address of the segment. The upper 4 bits are masked off.

Data Cache

The R4300 CPU has an 8 KB writeback data cache. This means that when the CPU writes a variable, it may not be written to main memory until later. Since the RSP reads the task list directly from main memory, the dynamic portion of the task list must be flushed from the data cache before the RSP starts.